

实验八：文件系统

练习 1：完成读文件操作的实现

一、实验目的

本练习要求在 Simple File System (SFS) 中实现文件读写核心函数 `sfs_io_nolock`，理解：

- inode 与磁盘数据块之间的映射关系；
- 非对齐读写与整块读写的处理方法；
- 文件大小变化与 inode 状态维护机制。

二、代码实现位置

实现代码位于：

```
kern/fs/sfs/sfs_inode.c
```

函数原型如下：

```
static int  
sfs_io_nolock(struct sfs_fs *sfs, struct sfs_inode *sin,  
              void *buf, off_t offset, size_t *alenp, bool write);
```

三、实现思路概述

由于 SFS 以 **block (4KB)** 为最小存储单位，而用户读写请求可能：

- 从任意偏移 `offset` 开始；
- 读写长度不固定；
- 跨越多个 block。

因此，文件读写过程被划分为三种情况分别处理：

- 起始非对齐部分
- 中间整块部分
- 结尾非对齐部分

四、核心代码与分析

4.1 起始非对齐部分

```
if ((blkoff = offset % SFS_BLKSIZE) != 0) {
    size = (nblks != 0) ? (SFS_BLKSIZE - blkoff) : (endpos - offset);

    if ((ret = sfs_bmap_load_nolock(sfs, sin, blkno, &ino)) != 0) {
        goto out;
    }

    if ((ret = sfs_buf_op(sfs, buf, size, ino, blkoff)) != 0) {
        goto out;
    }

    alen += size;
    if (nblks == 0) {
        goto out;
    }
    buf += size;
    blkno++;
    nblks--;
}
```

说明：

- 当 `offset` 未与 block 边界对齐时，需先处理当前 block 的剩余部分；
- 通过 `sfs_bmap_load_nolock` 将逻辑块号映射为磁盘块号；
- 使用 `sfs_rbuf` / `sfs_wbuf` 进行块内偏移读写。

4.2 中间整块读写

```
while (nblks > 0) {
    if ((ret = sfs_bmap_load_nolock(sfs, sin, blkno, &ino)) != 0) {
        goto out;
    }

    if ((ret = sfs_block_op(sfs, buf, ino, 1)) != 0) {
        goto out;
    }

    alen += SFS_BLKSIZE;
    buf += SFS_BLKSIZE;
    blkno++;
    nblks--;
}
```

说明：

- 对于完全对齐的 block，直接进行整块读写；

- 使用 `sfs_rblock` / `sfs_wblock` 提高 I/O 效率。

4.3 结尾非对齐部分

```
if ((size = endpos % SFS_BLKSIZE) != 0) {
    if ((ret = sfs_bmap_load_nolock(sfs, sin, endpos / SFS_BLKSIZE, &ino)) != 0) {
        goto out;
    }

    if ((ret = sfs_buf_op(sfs, buf, size, ino, 0)) != 0) {
        goto out;
    }

    alen += size;
}
```

说明：

- 当结束位置未对齐 block 时，处理最后一个 block 的部分内容；
- 从 block 起始位置进行读写。

4.4 文件大小更新

```
*alenp = alen;
if (offset + alen > sin->din->size) {
    sin->din->size = offset + alen;
    sin->dirty = 1;
}
```

- 写操作可能扩展文件大小；
- 更新 inode 的 `size` 字段并标记为 `dirty`，保证写回磁盘。

五、练习 1 验证方式

在用户态 shell 中执行：

```
hello > out
cat out
```

输出内容正确，说明文件读写功能实现成功。

练习 2：完成基于文件系统的执行程序机制

一、实验目标

本练习将程序加载方式由“内存加载”改为“文件系统加载”，实现：

- 从磁盘读取 ELF 可执行文件；
- 构建新的用户态虚拟地址空间；
- 正确执行用户程序。

二、load_icode_read 辅助函数

```
static int
load_icode_read(int fd, void *buf, size_t len, off_t offset)
{
    int ret;
    if ((ret = sysfile_seek(fd, offset, LSEEK_SET)) != 0) {
        return ret;
    }
    if ((ret = sysfile_read(fd, buf, len)) != len) {
        return (ret < 0) ? ret : -1;
    }
    return 0;
}
```

说明：

- 封装文件系统随机读取操作；
- 用于 ELF 头、程序头和段内容的加载。

三、用户栈构造与参数传递

用户栈布局如下：

```
+-----+
| argv 字符串内容 |
+-----+
| argv 指针数组   |
+-----+
| argc            |
+-----+
```

关键实现代码：

```
uint32_t argv_size = 0;
for (i = 0; i < argc; i++) {
    argv_size += strlen(kargv[i], EXEC_MAX_ARG_LEN + 1) + 1;
}
```

```
uintptr_t stacktop = USTACKTOP - ROUNDUP(argv_size, sizeof(long));
char **uargv = (char **)(stacktop - argc * sizeof(char *));

argv_size = 0;
for (i = 0; i < argc; i++) {
    uargv[i] = (char *)(stacktop + argv_size);
    strcpy(uargv[i], kargv[i]);
    argv_size += strlen(kargv[i], EXEC_MAX_ARG_LEN + 1) + 1;
}

stacktop = (uintptr_t)uargv - sizeof(int);
*(int *)stacktop = argc;
tf->gpr.sp = stacktop;
```

四、练习 2 验证方式

启动系统：

```
make qemu
```

在 shell 中执行：

```
hello
exit
```

程序能够正确运行，说明基于文件系统的 exec 机制实现成功。

五、实验总结

通过实验八的两个练习，成功实现了：

- SimpleFS 文件的完整读写流程；
- inode 与磁盘 block 的映射机制；
- 基于文件系统的 ELF 程序加载与执行；
- 文件系统、虚拟内存和进程管理模块之间的协同工作。

扩展练习

Challenge 1: Pipe 机制（数据结构与接口）

1) Pipe 核心对象: `upipe_t`

```
#define UPIPE_BUF_SIZE 4096
#define UPIPE_PIPE_BUF 4096 // 用于“小写原子性”(<=PIPE_BUF) 的阈值

typedef struct upipe {
    // 并发控制
    spinlock_t lock;           // 保护下面所有共享字段
    wait_queue_t rd_wait;      // 没数据时读端在这里睡眠
    wait_queue_t wr_wait;      // 没空间时写端在这里睡眠

    // 环形缓冲区
    uint8_t buf[UPIPE_BUF_SIZE];
    size_t head;               // 写入位置
    size_t tail;               // 读出位置
    size_t count;              // 当前已有字节数

    // 端点计数
    int readers;               // 打开的读端数量
    int writers;               // 打开的写端数量
    int refcnt;                // 被多少 file 引用（可选，但很实用）
} upipe_t;
```

解析:

`upipe_t` 是管道的本体，读端和写端会共享同一个 `upipe_t`。`buf/head/tail/count` 组成一个环形缓冲区：写端把数据放进 `buf`，读端从 `buf` 取走数据。`lock` 用来保证多个进程/线程同时读写时不会把 `head/tail/count` 等共享状态弄乱；当缓冲区“空”或“满”时，读写双方分别在 `rd_wait/wr_wait` 上睡眠，等对方写入/读出后再被唤醒。`readers/writers` 用于判断是否该阻塞还是返回 `EOF/EPIPE`；`refcnt` 用于最后释放管道对象。

2) file 私有信息: `upipe_file_priv_t`

```
typedef struct upipe_file_priv {
    upipe_t *p;
    bool can_read;             // 这个 fd 是否允许读
    bool can_write;            // 这个 fd 是否允许写
    bool nonblock;             // O_NONBLOCK 语义
} upipe_file_priv_t;
```

解析:

管道会返回两个 `fd`，一个只读一个只写。为了让同一个 `struct file` 知道自己是哪一端、是否是非阻塞模式，需要一个“私有信息结构”。这里 `p` 指向共享的 `upipe_t`；`can_read/can_write` 区分读端/写端，避免写端误读或读端误写；`nonblock` 用来决定“空/满时是睡眠等待还是立刻返回 `-EAGAIN`”。

3) 系统调用接口: `sys_pipe` / `sys_pipe2`

```
int sys_pipe(int fd[2]);
int sys_pipe2(int fd[2], int flags); // 支持 O_NONBLOCK / O_CLOEXEC
```

解析:

`sys_pipe` 的职责是“创建管道并安装 fd”: 分配并初始化一个 `upipe_t`, 再创建两个 `struct file` (一个读端、一个写端), 分别把 `upipe_file_priv_t` 挂上去 (指向同一个 `upipe_t`, 但权限相反), 最后放入当前进程的 fd 表并把 fd 编号写回 `fd[2]`。`sys_pipe2` 则是在创建时把 `flags` (尤其是 `O_NONBLOCK`) 记录到 `upipe_file_priv_t.nonblock` 中, 从而影响后续 read/write 的阻塞行为。

4) pipe 专用 `file_ops`:

`pipe_read/pipe_write/pipe_close/...`

```
ssize_t pipe_read(struct file *f, void *buf, size_t n);
ssize_t pipe_write(struct file *f, const void *buf, size_t n);
int pipe_close(struct file *f);
int pipe_fcntl(struct file *f, int cmd, unsigned long arg); // 切换 nonblock 等
int pipe_poll(struct file *f, int events); // select/poll
```

解析:

`pipe_read` 在缓冲区有数据时拷贝出去并推进 `tail/count`; 若缓冲区为空则根据 `writers` 和 `nonblock` 决定阻塞等待、返回 EOF (写端全关) 或返回 `-EAGAIN`。

`pipe_write` 在缓冲区有空间时写入并推进 `head/count`; 若缓冲区满则根据 `readers` 和 `nonblock` 决定阻塞等待、返回 `-EAGAIN` 或返回 `-EPIPE` (读端全关)。

`pipe_close` 负责更新 `readers/writers/refcnt` 并在端点消失时唤醒对方 (让阻塞的读/写能及时退出)。

`pipe_fcntl/pipe_poll` 属于增强能力: 切换非阻塞、支持 select/poll 等。

Challenge 2: 硬链接 + 软链接 (数据结构与接口)

1) inode 结构: 增加 `i_nlink` 与软链接内容

```
#define S_IFREG 0100000
#define S_IFDIR 0040000
#define S_IFLNK 0120000

typedef struct inode {
    uint32_t i_no;
    uint16_t i_mode; // 至少能区分 REG/DIR/LNK
    uint32_t i_size;
    uint32_t i_nlink; // 硬链接计数
    atomic_t i_ref;
    spinlock_t i_lock; // 保护 i_nlink/i_size/元数据
```

```

union {
    struct {
        void *data_mapping;
    } reg;
    struct {
        char *target;    // 软链接内容：指向的路径字符串
        uint32_t target_len;
    } lnk;
} u;

const struct inode_ops *i_ops; // 类 Linux VFS 的 inode 操作表
} inode_t;

```

解析：

硬链接的本质是“多个名字指向同一个inode”，因此必须有 `i_nlink` 来记录“有多少目录项正在指向我”。而 `i_ref` 用来记录运行时还有没有人打开/引用这个 inode，用于实现 unlink 的延迟删除：名字删了但仍被打开时不能立刻回收。

软链接本质上不是“指向同一个 inode”，而是一个特殊类型的 inode（`S_IFLNK`），它的内容是 `target` 字符串，所以用 `u.lnk.target/target_len` 来保存。

`i_lock` 用于保护这些计数与元数据，防止并发 link/unlink 导致计数错乱。

2) 目录项结构：`dirent_t`

```

#define NAME_MAX 255

typedef struct dirent {
    uint32_t ino;        // 指向的 inode number
    uint8_t type;        // DT_REG/DT_DIR/DT_LNK...
    char name[NAME_MAX + 1];
} dirent_t;

```

解析：

在某个目录里再加一条新的 `dirent_t`，让它的 `ino` 指向原文件同一个 inode；而 unlink 则是把某条 `dirent_t` 删除。`type` 能帮助路径解析与工具判断该名字对应的是普通文件、目录还是软链接。

3) 目录修改锁：`dir_lock_t`

```

typedef struct dir_lock {
    mutex_t mtx;        // 谁改目录（增删改 dirent）谁先拿它
} dir_lock_t;

```

解析：

目录是共享的名字空间，多进程可能同时在同一目录下创建/删除文件名。如果没有目录锁，两个线程同时“插入/删除 dirent”很容易把目录内容写乱。因此对目录的结构化修改（link/unlink/symlink/mkdir/create等）都必须先拿 `dir_lock_t.mtx`，做到“目录更新串行化”。这是一种最小但有效的同步设计。

4) 系统调用接口: `sys_link/sys_unlink/sys_symlink/...`

```
int sys_link(const char *oldpath, const char *newpath); // 硬链接: 新增目录项指向同
inode
int sys_unlink(const char *path); // 删除目录项: i_nlink--
int sys_symlink(const char *target, const char *linkpath); // 软链接: 新建 LNK
inode, 内容=target
ssize_t sys_readlink(const char *path, char *buf, size_t bufsz); // 读出软链接内容 (不跟随)
int sys_lstat(const char *path, struct stat *st); // 可选: 不跟随 symlink 的
stat
```

解析:

`sys_link`: 找到 `oldpath` 对应 `inode`, 然后在 `newpath` 的父目录里新增一条目录项指向该 `inode`, 并在 `inode` 锁保护下 `i_nlink++`。

`sys_unlink`: 删除 `path` 对应的目录项, 并对 `inode` 做 `i_nlink--`; 如果 `i_nlink==0` 且 `i_ref==0` 才真正回收数据, 否则等最后关闭 (`i_ref` 归零) 再回收。

`sys_symlink`: 在父目录中新建一个类型为 `S_IFLNK` 的 `inode`, 把 `target` 字符串写进 `inode->u.lnk`, 再建立目录项。

`sys_readlink`: 读取软链接 `inode` 中保存的 `target` 字符串返回给用户 (注意是不跟随)。

`sys_lstat`: 用于“看见软链接本身”, 而不是跟随到 `target`。

5) VFS/inode操作表: `inode_ops`

```
struct inode_ops {
    int (*lookup)(inode_t *dir, const char *name, inode_t **out);
    int (*link)(inode_t *dir, const char *name, inode_t *target); // 硬链接:
dir/name -> target
    int (*unlink)(inode_t *dir, const char *name); // 删目录项 + 更
新 i_nlink
    int (*symlink)(inode_t *dir, const char *name, const char *target); // 创建 LNK
inode + 写 target
    ssize_t (*readlink)(inode_t *inode, char *buf, size_t bufsz); // 从 LNK inode
读 target
};
```

解析:

路径解析时先 `lookup` 找到名字对应的 `inode`; 硬链接/删除链接/创建软链接分别对应 `link/unlink/symlink`; 当遇到软链接 `inode` 时, 通过 `readlink` 拿到 `target` 字符串继续解析路径。这样做的好处是: 系统调用不需要关心底层目录如何存储 `dirent`、`inode` 如何落盘, 只要调用统一的 `ops` 即可。